National University of Singapore
School of Computing
CS3244: Machine Learning
Solution to Tutorial 6 (1 week; in class)

Decision Trees and Ensemble Methods

Readings: *CIML, Section 1.3 & Chapter 11.*

1. **Decision Trees.**

   The loans department of DBN (Development Bank of NUS) has the following past loan processing records each containing an applicants income, credit history, debt, and the final approval decision.

Income	Credit History	Debt	Decision
0 – 5K	Bad	Low	Reject
0 – 5K	Good	Low	Approve
0 – 5K	Unknown	High	Reject
0 – 5K	Unknown	Low	Approve
0 – 5K	Unknown	Low	Approve
0 – 5K	Unknown	Low	Reject
5 – 10K	Bad	High	Reject
5 – 10K	Good	High	Approve
5 – 10K	Unknown	High	Approve
5 – 10K	Unknown	Low	Approve
Over 10K	Bad	Low	Reject
Over 10K	Good	Low	Approve

   (a) Construct a decision tree based on the above training examples. (Note: $\log_2 \frac{x}{y} = \log_2$ x - $\log_2$ y, $\log_2$ 1 = 0, $\log_2$ 2 = 1, $\log_2$ 3 = 1.585, $\log_2$ 4 = 2, $\log_2$ 5 = 2.322, $\log_2$ 6 = 2.585, $\log_2$ 7 = 2.807, $\log_2$ 8 = 3, $\log_2$ 9 = 3.170, $\log_2$ 10 = 3.322, $\log_2$ 11 = 3.459, and $\log_2$ 12 = 3.585)

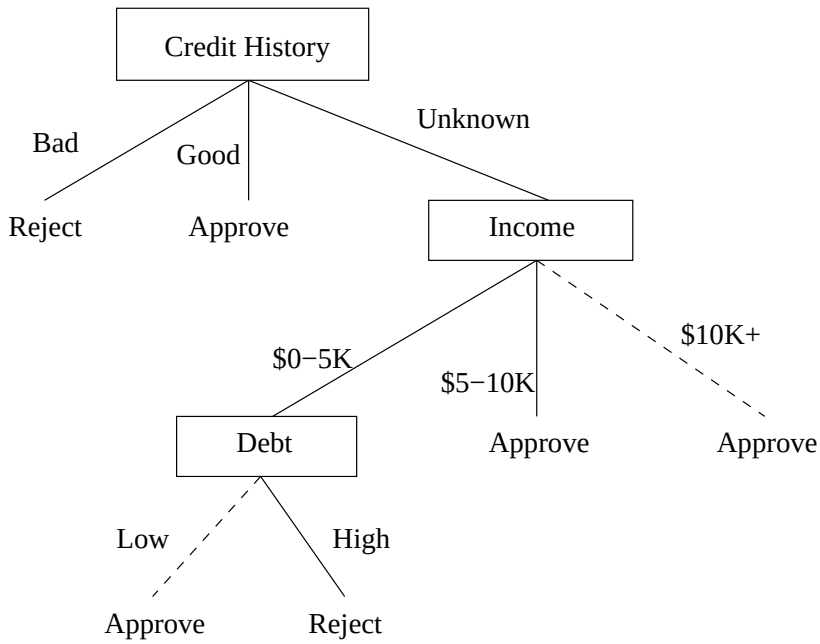
$$
\begin{aligned}
IG(Income) &= H(\frac{5}{12}, \frac{7}{12}) - remainder(Income) \\
remainder(Income) &= \frac{6}{12}(-\frac{3}{6}\log_2\frac{3}{6} - \frac{3}{6}\log_2\frac{3}{6}) + \frac{4}{12}(-\frac{1}{4}\log_2\frac{1}{4} \\
&\quad -\frac{3}{4}\log_2\frac{3}{4}) + \frac{2}{12}(-\frac{1}{2}\log_2\frac{1}{2} - \frac{1}{2}\log_2\frac{1}{2}) \\
&= 0.937 \\
IG(Income) &= 0.98 - 0.937 = 0.043 \\
remainder(CreditHistory) &= \frac{3}{12}(-\frac{3}{3}\log_2\frac{3}{3} - \frac{0}{3}\log_2\frac{0}{3}) + \frac{3}{12}(-\frac{3}{3}\log_2\frac{3}{3} \\
&\quad -\frac{0}{3}\log_2\frac{0}{3}) + \frac{6}{12}(-\frac{2}{6}\log_2\frac{2}{6} - \frac{4}{6}\log_2\frac{4}{6}) \\
&= 0.459 \\
IG(CreditHistory) &= 0.98 - 0.459 = 0.521 \\
remainder(Debt) &= \frac{8}{12}(-\frac{3}{8}\log_2\frac{3}{8} - \frac{5}{8}\log_2\frac{5}{8}) + \frac{4}{12}(-\frac{2}{4}\log_2\frac{2}{4} - \frac{2}{4}\log_2\frac{2}{4}) \\
&= 0.970 \\
IG(Debt) &= 0.98 - 0.970 = 0.01
\end{aligned}
$$

Since Credit History has the highest gain, choose it as the root, which has three values, i.e., "Bad", "Good", and "Unknown". Since all examples for "Bad" have the same classification (i.e., "Reject") and all examples for "Good" have the same classification (i.e., "Approve"), both nodes have no further subtree. For "Unknown", a subtree for the following subset of examples is to be constructed:

Income	*Debt*	*Decision*
$0-5K$	*High*	*Reject*
$0-5K$	*Low*	*Approve*
$0-5K$	*Low*	*Approve*
$0-5K$	*Low*	*Reject*
$5-10K$	*High*	*Approve*
$5-10K$	*Low*	*Approve*

$$
\begin{aligned}
H(\frac{2}{6}, \frac{4}{6}) &= -\frac{2}{6}\log_2\frac{2}{6} - \frac{4}{6}\log_2\frac{4}{6} = 0.918 \\
remainder(Income) &= \frac{4}{6}(-\frac{2}{4}\log_2\frac{2}{4} - \frac{2}{4}\log_2\frac{2}{4}) + \frac{2}{6}(-\frac{2}{2}\log_2\frac{2}{2} - \frac{0}{2}\log_2\frac{0}{2}) = 0.667 \\
gain(Income) &= 0.918 - 0.667 = 0.251 \\
remainder(Debt) &= \frac{2}{6}(-\frac{1}{2}\log_2\frac{1}{2} - \frac{1}{2}\log_2\frac{1}{2}) + \frac{4}{6}(-\frac{1}{4}\log_2\frac{1}{4} - \frac{3}{4}\log_2\frac{3}{4}) = 0.874 \\
gain(Debt) &= 0.918 - 0.874 = 0.044
\end{aligned}
$$

Since Income has a higher gain than Debt, Income is chosen as the root of the subtree under Credit History=Unknown.



The two decisions on dotted lines are those that are not quite so straight forward. In one case, there is insufficient data for us to decide what to do, so we do something reasonable. In the other case, the resulting sample is not decisive. There are two approves and one reject. We can use a simple majority to decide.

Question is: why do we have such a node where there seems to be ambiguity? There is no clear answer (or at least we don't have the information to determine). One possibility is that we are missing some attribute from the data that will allow us to differentiate at this last node. Another possibility is that there is some non-determinism in the underlying function that we are trying to approximate with this decision tree.

(b) **Adaboost + Decision Stumps.** Construct the Adaboost series of decision stumps given the same set of examples. How many iterations does Adaboost take before terminating?

We will only do 2 rounds of boosting, in the below analysis as it illustrates the necessary points.

In the first round, Adaboost will choose the same decision stump as the complete decision tree, choosing Credit History as the attribute for the decision, as all of the examples have the same weight, $\frac{1}{12}$. *Note that the Credit History == "Unknown" defaults to "Approve" since* $\frac{4}{6}$ *of the examples indicate "Approve". The classifier gets in total* $\frac{10}{12}$ *correct, missing the 3rd and 6th examples in the original table, hence* $\epsilon_1 = \frac{1}{6}$. α_1 *(the weight for this first classifier)* $= \frac{1}{2}ln(\frac{1-\frac{1}{6}}{\frac{1}{6}}) = \frac{1}{2}ln5 \approx .80$. *Then for the next round (2), we calculate* $exp(-\alpha_1 y_n g_1(\mathbf{x}_n))$, *where we take* $y_n g_1(\mathbf{x}_n) == 1$ *for examples that are correct and -1 for examples that are incorrect. Weights for Round 2 are thus* $\mathcal{D}_2 = \frac{1 \cdot e^{-.8}}{Z_1} \approx \frac{0.45}{Z_1} = 0.05$ *for correct examples,* $\approx \frac{2.22}{Z_1} = 0.25$ *for incorrect examples (before and after* $Z_1 = 8.94$ *normalization over all 12 points).*

In Round 2, we must redo the information gain calculation in the face of the new weights for the examples. At the root now, we have: $H(\mathcal{D}) \approx H(0.65, 0.35) \approx 0.93$

$$\begin{aligned}
R_{Income} &= 0.7 \times H(0.21, 0.79) + 0.2 \times H(0.25, 0.75) + 0.1 \times H(0.5, 0.5) \\
&= 0.7 \times 0.74 + 0.2 \times 0.81 + 0.1 \times 1.0 = 0.52 + 0.16 + 0.08 = 0.76 \\
IG_{Income} &= 0.93 - 0.76 = 0.17 \\
R_{CreditHistory} &= 0.15 \times H(1, 0) + 0.15 \times H(0, 1) + 0.7 \times H(0.44, 0.55) \\
&= 0 + 0 + 0.7 \times 0.99 = 0.69 \\
IG_{CreditHistory} &= 0.93 - 0.69 = 0.24 \\
R_{Debt} &= 0.6 \times H(.58, .41) + 0.4 \times H(.75, .25) \\
&= 0.6 \times 0.98 + 0.4 \times 0.81 = 0.59 + 0.32 = 0.91 \\
IG_{Debt} &= 0.98 - 0.91 = 0.07
\end{aligned}$$

where in the above, "R" denotes remainder, and "IG" denotes information gain.

With the changes in weighting, we see that the information gain of Credit History has decreased a bit, and that both Debt and Income have increased. However, with one step, the weights have not sufficiently changed to change the root (decision stump)'s choice of attribute to split on, and we can see that the classifier still picks Credit History, and gets the same misclassification rate $\epsilon_2 = \frac{1}{6}$.

This would be further perturbed in later rounds of the boosting algorithm and would eventually select the Income attribute to split on, choosing to "Reject" cases with "0 – 5K" income as that would get the two incorrect examples (#3 and #6) correct.

In reality, we need to define a termination criterion that will allow the algorithm to terminate. As with other iterative algorithms, we can see that the classification of the points has not changed over two iterations in this example, yet the weights on the points are significantly changing. As such it is often useful **not** *to set the termination criterion that is only based on the stability of the classification (as the weights can change over several iterations that will eventually result in a change in classification).*

(c) **Random Forest.** Construct a random forest of size 3, where each of the three decision trees is fully grown from two of the three attributes, again based on the same set of examples: {Income, Credit History},{Credit History, Debt} and {Debt, Income}.

Based on the earlier answers for the decision tree, we know that in the first two examples, Credit History will be the root node, due to its better information gain. This leads to pure "Bad" and "Good" leaves, leading to only using the secondary attribute for the remaining six cases of "Unknown".

For the {Income, Credit History} tree, it is identical to the one for the full decision tree, except that the bottom Debt node is left off and replaced by the mode of the examples that

fulfill the tests (Credit History == "Unknown" ∩ Income == "0 – 5K"). This is a tie, so the parent node's test of just Credit History == "Unknown" is used to obtain "Approve".

For the {Credit History, Debt} tree, the Credit History == "Unknown" takes a Debt test, resulting "Approve" for "Low" values (by the mode of examples), and also "Approve" for "High" values (by the mode of the parent, since the mode of the examples is tied).

For the {Debt, Income} tree, we can recall that the information gain of the previous decision tree showed that Income was better than Debt. All three subcases of Income are not pure and each needs a test for Debt. In the "0 – 5K" Income, Debt == "Low" yields "Approve" by the mode of the examples, and yields "Reject" for "High" Debt cases. In the "5 – 10K" Income, Debt == "High" yields "Approve" by the mode of the examples, and yields "Approve" by purity when Debt is "Low". In the "Over 10K" Income, there are no examples of "High", so it defaults to "Approve". Similarly, since there is a tied outcome for Debt == "Low", we default to the mode of all examples, hence "Approve".

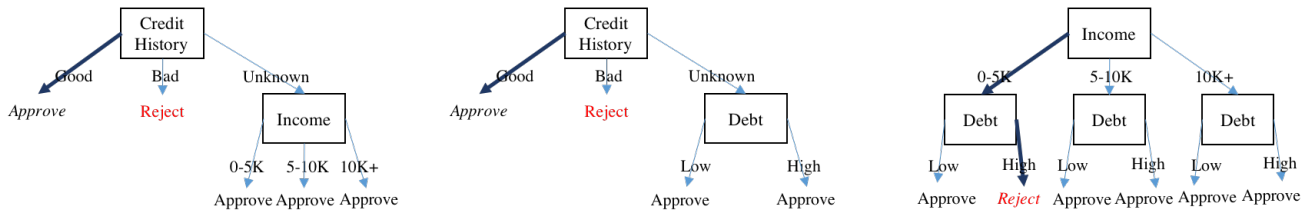


Figure 1: Random Forest of size 3 in Part (c). The dark edges and italicized labels correspond to the decision for each tree for the individual described in Part (d).

(d) What is the decision tree classifier's decision for a person who has 4K yearly income, a good credit history and a high amount of debt? Are your results different for the Adaboost + Decision Stumps in Part (b). or the Random Forest in Part (c)?

For the individual, the decision tree approves of the application from the root node of the tree.

For the Adaboost-Decision Stumps of two rounds, the decision stump is the same in both rounds, and also identical to the decision tree. Hence it would accept the applicant. Note that both rounds have the same voting power as their misclassification rate is identical.

For the random forest of size 3, as done in Part (c), the first two trees vote "Approve", but the final {Income, Debt} tree votes "Reject" (as it satisfies the Income =="0 – 5K" → Debt == "High" → "Reject" path. Random Forests employs uniform voting, so the $\frac{2}{3}$rds vote wins to "Approve" the applicant too.

2. **Uniform Blending.** One of the simplest ensemble methods is Uniform Blending. Given a set of hypothesis: $g_1, g_2, g_3, ..., g_T$, Uniform blending makes predictions simply by mixing the

predictions given by $g_1, g_2, g_3, ..., g_T$ uniformly. Concretely, for binary classification, uniform blending predicts by:

$$G(x) = \text{sign}(\sum_{t=1}^{T} g_t(x)), \tag{1}$$

and for regression, uniform blending predicts by:

$$G(x) = \frac{1}{T}\sum_{t=1}^{T} g_t(x) \tag{2}$$

Take regression as the example, prove that the performance (measured by out-of-sample error) of uniform blending is no worse than the average performance over $g_1, g_2, g_3, ..., g_T$. Assume we evaluate the testing error by mean square error. (By proving this, it can give us an intuition on why ensembling can help to reduce the out-of-sample error)

Let's first see the case for only one fixed data point x. Suppose the ground truth function is $f(x)$. The average error over $g_1, g_2, g_3, ..., g_T$ for point x is

$$\begin{aligned}
\frac{1}{T}\sum_{t=1}^{T}[g_t(x) - f(x)]^2 &= \frac{1}{T}\sum_{t=1}^{T}\left[g_t(x)^2 - 2g_t(x)f(x) + f(x)^2\right] && (3)\\
&= \frac{1}{T}\sum_{t=1}^{T} g_t(x)^2 - \frac{2f(x)}{T}\sum_{t=1}^{T} g_t(x) + \frac{1}{T}\sum_{t=1}^{T} f(x)^2 && (4)\\
&= \frac{1}{T}\sum_{t=1}^{T} g_t(x)^2 - 2f(x)G(x) + \frac{1}{T}\sum_{t=1}^{T} f(x)^2 \qquad G(x) = \tfrac{1}{T}\textstyle\sum_{t=1}^{T} g_t(x) && (5)\\
&= \frac{1}{T}\sum_{t=1}^{T} g_t(x)^2 - 2f(x)G(x) + f(x)^2 \qquad f(x) \text{ does not depend on } t && (6)\\
&= \frac{1}{T}\sum_{t=1}^{T} g_t(x)^2 - G(x)^2 + G(x)^2 - 2f(x)G(x) + f(x)^2 && (7)\\
&= \frac{1}{T}\sum_{t=1}^{T} g_t(x)^2 - G(x)^2 + [G(x) - f(x)]^2 && (8)\\
&= \frac{1}{T}\sum_{t=1}^{T} g_t(x)^2 - 2G(x)^2 + G(x)^2 + [G(x) - f(x)]^2 && (9)\\
&= \frac{1}{T}\sum_{t=1}^{T} g_t(x)^2 - \frac{1}{T}\sum_{t=1}^{T} 2g_t(x)G(x) + \frac{1}{T}\sum_{t=1}^{T} G(x)^2 + [G(x) - f(x)]^2 && (10)\\
&= \frac{1}{T}\sum_{t=1}^{T}\left[g_t(x)^2 - 2g_t(x)G(x) + G(x)^2\right] + [G(x) - f(x)]^2 && (11)\\
&= \frac{1}{T}\sum_{t=1}^{T}[g_t(x) - G(x)]^2 + [G(x) - f(x)]^2 && (12)
\end{aligned}$$

To compute the out-of-sample error, we only need to compute the expected error over all x, hence, the average out-of-sample error over $g_1, g_2, g_3, ..., g_T$ is

$$\frac{1}{T}\sum_{t=1}^{T}\mathbb{E}_{out}(g_t(x)) = \frac{1}{T}\sum_{t=1}^{T}\mathbb{E}_x\left\{[g_t(x)-f(x)]^2\right\} \tag{13}$$

$$= \mathbb{E}_x\left\{\frac{1}{T}\sum_{t=1}^{T}[g_t(x)-f(x)]^2\right\} \tag{14}$$

$$= \mathbb{E}_x\left\{\frac{1}{T}\sum_{t=1}^{T}[g_t(x)-G(x)]^2+[G(x)-f(x)]^2\right\} \tag{15}$$

$$= \mathbb{E}_x\left\{\frac{1}{T}\sum_{t=1}^{T}[g_t(x)-G(x)]^2\right\}+\mathbb{E}_x\left\{[G(x)-f(x)]^2\right\} \tag{16}$$

$$= \mathbb{E}_x\left\{\frac{1}{T}\sum_{t=1}^{T}[g_t(x)-G(x)]^2\right\}+\mathbb{E}_{out}(G(x)) \tag{17}$$

$$\geq \mathbb{E}_{out}(G(x)) \tag{18}$$

From the above deduction we can see, the average out-of-sample error over $g_1, g_2, g_3, ..., g_T$ is greater than or equal to the out-of-sample error of $G(x)$. Hence, the performance (measured by out-of-sample error) of uniform blending is no worse than the average performance over $g_1, g_2, g_3, ..., g_T$.

3. **Adaptive Boosting.** Algorithm 1 shows AdaBoost Algorithm for binary classification:

Algorithm 1 AdaBoost for Binary Classification

1: Initialize the data points' weights to $w^{(1)} = [\frac{1}{N}, \frac{1}{N}, ..., \frac{1}{N}]$
2: **for** $t = 1, 2, ..., T$ **do**
3: Fit a classifier $g_t(x)$ to the training data with weights $w^{(t)}$
4: Compute the $w^{(t)}$-weighted 0/1 error ϵ_t by:

$$\epsilon_t = \frac{\sum_{i=1}^{N} w_i^{(t)} I(y_i \neq g_t(x_i))}{\sum_{i=1}^{N} w_i^{(t)}}$$

where $I(statement)$ is a function that output 1 if *statement* is true, 0 otherwise.
5: Compute $\alpha_t = \frac{1}{2}\ln(\frac{1-\epsilon_t}{\epsilon_t})$
6: Set $w_i^{(t+1)} \leftarrow \frac{w_i^{(t)}\exp[-\alpha_t y_i g_t(x_i)]}{Z_t}$, $i = 1, 2, ..., N$, where Z_t is the normalizer in t-th iteration.
7: **end for**
8: **return** $G(x) = \text{sign}[\sum_{t=1}^{T}\alpha_t g_t(x)]$

(a) Show that the normalizer Z_t in Line 6 is actually not necessary, i.e., we can update the weight by $w_i^{(t+1)} \leftarrow w_i^{(t)}\exp[-\alpha_t y_i g_t(x_i)], i = 1, 2, ..., N$.

When training $g_{t+1}(x)$ with the weighted error, The loss function is:

$$\frac{1}{N}\sum_{i=1}^{N} w_i^{(t+1)} I(y_i \neq g_{t+1}(x_i)) \tag{19}$$

$$= \frac{1}{N}\sum_{i=1}^{N} \frac{w_i^{(t)} \exp[-\alpha_t y_i g_t(x_i)]}{Z_t} I(y_i \neq g_{t+1}(x_i)) \tag{20}$$

$$= \frac{1}{NZ_t}\sum_{i=1}^{N} w_i^{(t)} \exp[-\alpha_t y_i g_t(x_i)] I(y_i \neq g_{t+1}(x_i)) \tag{21}$$

$$g_{t+1}^* = \arg\min_{g_{t+1}} \frac{1}{N}\sum_{i=1}^{N} w_i^{(t+1)} I(y_i \neq g_{t+1}(x_i)) \tag{22}$$

$$= \arg\min_{g_{t+1}} \frac{1}{NZ_t}\sum_{i=1}^{N} w_i^{(t)} \exp[-\alpha_t y_i g_t(x_i)] I(y_i \neq g_{t+1}(x_i)) \tag{23}$$

$$= \arg\min_{g_{t+1}} \frac{1}{N}\sum_{i=1}^{N} w_i^{(t)} \exp[-\alpha_t y_i g_t(x_i)] I(y_i \neq g_{t+1}(x_i)) \tag{24}$$

The last equation holds because Z_t is a normalization constant. It does not affect the selection of g_{t+1}.

From the above equations we can see, g_{t+1}^* is the same without the normalizer Z_t. Hence omitting Z_t in weight updating process won't have an effect.

(b) Exponential Loss is defined as follows:

$$L(\mathbf{Y}, g(\mathbf{X})) = \frac{1}{N}\sum_{i=1}^{N} \exp(-y_i g(x_i)) \tag{25}$$

Define a stagewise additive function as follows:

$$C_t(x) = \alpha_1 g_1(x) + \alpha_2 g_2(x) + ... + \alpha_t g_t(x) \tag{26}$$

$$= C_{t-1}(x) + \alpha_t g_t(x) \tag{27}$$

where $C_{t-1}(x) = \alpha_1 g_1(x) + \alpha_2 g_2(x) + ... + \alpha_{t-1} g_{t-1}(x)$ is recursively defined, and $C_0(x) = 0$, and each $g_j(x)$ is a binary classifier and each $\alpha_j > 0$. Now, given $C_{t-1}(x)$, we want to decide the optimal α_t and $g_t(x)$ that minimizes the exponential loss:

$$L(\mathbf{Y}, C_t(\mathbf{X})) \tag{28}$$

Prove that computing α_t and $g_t(x)$ that minimizes $L(\mathbf{Y}, C_t(\mathbf{X}))$ is equivalent to computing the α_t and $g_t(x)$ in AdaBoost shown in Algorithm 1 (Hence AdaBoost is equivalent to forward stagewise additive modeling using the exponential loss function).

The exponential loss is:

$$L(\mathbf{Y}, C_t(\mathbf{X})) = \frac{1}{N}\sum_{i=1}^{N} \exp(-y_i C_t(x_i)) \tag{29}$$

$$= \frac{1}{N}\sum_{i=1}^{N} \exp(-y_i C_{t-1}(x_i) - y_i \alpha_t g_t(x_i)) \tag{30}$$

$$= \frac{1}{N}\sum_{i=1}^{N} \exp(-y_i C_{t-1}(x_i)) \cdot \exp(-y_i \alpha_t g_t(x_i)) \tag{31}$$

where α_t and g_t are yet to be determined to minimize the exponential loss. Since we are minimizing exponential loss by optimizing α_t and $g_t(x)$ and $\exp(-y_i C_{t-1}(x_i))$ does not contain either α_t or $g_t(x)$. We can extract it out. Define:

$$w_i^{(t)} = \exp(-y_i C_{t-1}(x_i)), \tag{32}$$

and define $w_i^{(1)} = 1$, then:

$$w_i^{(t)} = \exp(-y_i C_{t-1}(x_i)) \tag{33}$$

$$= \exp\{-y_i[C_{t-2}(x_i) + \alpha_{t-1} g_{t-1}(x_i)]\} \tag{34}$$

$$= \exp(-y_i(C_{t-2}(x_i)) \cdot \exp(-\alpha_{t-1} y_i g_{t-1}(x_i)) \tag{35}$$

$$= w_i^{(t-1)} \cdot \exp(-\alpha_{t-1} y_i g_{t-1}(x_i)) \tag{36}$$

We have proved in (a) that normalization on the weight is not necessary. Hence the weight updating process in Equation 36 is the same with that in Line 6, Algorithm 1. Applying Equation 32 to the loss function, we get:

$$L(\mathbf{Y}, C_t(\mathbf{X})) = \frac{1}{N}\sum_{i=1}^{N} w_i^{(t)} \cdot \exp(-y_i \alpha_t g_t(x_i)) \tag{37}$$

Since $g_t(x)$ is a binary classifier, $y_i g_t(x_i) = 1$ if $y_i = g_t(x_i)$, -1 otherwise. Hence we can rewrite Equation 37 into

$$L(\mathbf{Y}, C_t(\mathbf{X})) = \frac{1}{N}\sum_{y_i = g_t(x_i)} w_i^{(t)} \cdot \exp(-\alpha_t) + \frac{1}{N}\sum_{y_i \neq g_t(x_i)} w_i^{(t)} \cdot \exp(\alpha_t) \tag{38}$$

To get α_t that minimizes the exponential loss, we compute the partial differential of $L(\mathbf{Y}, C_t(\mathbf{X}))$ with respect to α_t:

$$\frac{\partial L(\mathbf{Y}, C_t(\mathbf{X}))}{\partial \alpha_t} = -\frac{1}{N} \sum_{y_i = g_t(x_i)} w_i^{(t)} \cdot \exp(-\alpha_t) + \frac{1}{N} \sum_{y_i \neq g_t(x_i)} w_i^{(t)} \cdot \exp(\alpha_t) \tag{39}$$

Let $\frac{\partial L(\mathbf{Y}, C_t(\mathbf{X}))}{\partial \alpha_t} = 0$, *we get*

$$\alpha_t = \frac{1}{2} \ln \left\{ \frac{\sum_{y_i = g_t(x_i)} w_i^{(t)}}{\sum_{y_i \neq g_t(x_i)} w_i^{(t)}} \right\} \tag{40}$$

$$= \frac{1}{2} \ln \left\{ \frac{\sum_{y_i = g_t(x_i)} w_i^{(t)} / \sum w_i^{(t)}}{\sum_{y_i \neq g_t(x_i)} w_i^{(t)} / \sum w_i^{(t)}} \right\} \tag{41}$$

$$= \frac{1}{2} \ln(\frac{1 - \epsilon_t}{\epsilon_t}) \tag{42}$$

From the above equation we can see, the α_t *that minimizes* $L(\mathbf{Y}, C_t(\mathbf{X}))$ *is exactly the same with the one in Line 5, Algorithm 1.*

For now, we have proved that the weight updating process and α_t *is the same with AdaBoost. If we can also prove that optimizing* $g_t(x)$ *to minimize* $L(\mathbf{Y}, C_t(\mathbf{X}))$ *is the same with optimizing* $g_t(x)$ *to get lowest error using the weighted training data in AdaBoost, we can conclude AdaBoost is equivalent to forward stagewise additive modeling using the exponential loss function. The following proves optimizing* $g_t(x)$ *to minimize* $L(\mathbf{Y}, C_t(\mathbf{X}))$ *is the same with optimizing* $g_t(x)$ *to get lowest error using the weighted training data in AdaBoost:*

$$L(\mathbf{Y}, C_t(\mathbf{X})) = \frac{1}{N} \sum_{y_i = g_t(x_i)} w_i^{(t)} \cdot \exp(-\alpha_t) + \frac{1}{N} \sum_{y_i \neq g_t(x_i)} w_i^{(t)} \cdot \exp(\alpha_t) \tag{43}$$

$$\Rightarrow \exp(\alpha_t) \cdot L(\mathbf{Y}, C_t(\mathbf{X})) = \frac{1}{N} \sum_{y_i = g_t(x_i)} w_i^{(t)} + \frac{1}{N} \sum_{y_i \neq g_t(x_i)} w_i^{(t)} \cdot \exp(2\alpha_t) \tag{44}$$

$$\Rightarrow \exp(\alpha_t) \cdot L(\mathbf{Y}, C_t(\mathbf{X})) = \frac{1}{N} \sum w_i^{(t)} - \frac{1}{N} \sum_{y_i \neq g_t(x_i)} w_i^{(t)} + \frac{1}{N} \sum_{y_i \neq g_t(x_i)} w_i^{(t)} \cdot \exp(2\alpha_t) \tag{45}$$

$$\Rightarrow \exp(\alpha_t) \cdot L(\mathbf{Y}, C_t(\mathbf{X})) = \frac{1}{N} \sum w_i^{(t)} + \frac{1}{N} \sum_{y_i \neq g_t(x_i)} w_i^{(t)} \cdot [\exp(2\alpha_t) - 1] \tag{46}$$

For the selection of $g_t(x)$, α_t *is irrelevant, and* $\frac{1}{N} \sum w_i^{(t)}$ *is also irrelevant with* $g_t(x)$. *Hence, to minimize* $L(\mathbf{Y}, C_t(\mathbf{X}))$, *it is equivalent to minimize* $\frac{1}{N} \sum_{y_i \neq g_t(x_i)} w_i^{(t)} \cdot [\exp(2\alpha_t) - 1]$.

Furthermore, by a similar argument, $\alpha_t > 0$, $[\exp(2\alpha_t) - 1] > 0$, *and so it is equivalent to minimize* $\frac{1}{N}\sum_{y_i \neq g_t(x_i)} w_i^{(t)}$. *So,*

$$g_t^* = \arg\min_{g_t} \frac{1}{N} \sum_{y_i \neq g_t(x_i)} w_i^{(t)} \tag{47}$$

$$= \arg\min_{g_t} \frac{\sum_{y_i \neq g_t(x_i)} w_i^{(t)}}{\sum w_i^{(t)}} \tag{48}$$

The expression inside $\arg\min$ *in the right hand side of Equation 48 is actually the weighted error rate in predicting* y *using* g_t *as the classifier. Hence computing* $g_t(x)$ *that minimizes* $L(\mathbf{Y}, C_t(\mathbf{X}))$ *is equivalent to optimizing* $g_t(x)$ *with lowest error using the weighted training data in AdaBoost.*